

# Zugänge

## Source Code

- <https://github.com/baernhaeck/SeniorsInTheMiddle>
  - /backend : Enthält den Forward-Proxy sowie die WebAPI und den Telemetry-Stream.
  - /frontend : Enthält das Dashboard, mit dem der Datenfluss beobachtet und geprüft wird.
  - /services : Enthält die Python-Services, die der Proxy über Unix-Sockets aufruft, inklusive PII-Erkennung.
  - /integration : Die Testumgebung, die den Proxy unverändert als Image betreibt und Verkehr durchschickt. Enthält zusätzlich den Demo-Browser.
  - /notebooks : Explorative Notebooks zur Erkennungsqualität.
  - /docs : Enthält die Dokumentation für die Jury
  - /pitch : Enthält die Slides für den Pitch sowie den Screencast

## Deployment

- Frontend: <https://seniorsinthemiddle-frontend.icymushroom-561b0fa4.northeurope.azurecontainerapps.io/>
- Backend: <https://seniorsinthemiddle-backend.icymushroom-561b0fa4.northeurope.azurecontainerapps.io/swagger>
- CI/CD Pipelines: <https://github.com/baernhaeck/SeniorsInTheMiddle/actions>
- Deployments: [https://github.com/orgs/baernhaeck/packages?repo\\_name=SeniorsInTheMiddle](https://github.com/orgs/baernhaeck/packages?repo_name=SeniorsInTheMiddle)

## Ausgangslage

Natron Tech betreibt Infrastruktur für Kunden, denen zugesagt wurde, dass bestimmte Daten die Schweiz nicht verlassen und einem ausländischen Anbieter technisch nie zu Gesicht kommen. Gleichzeitig laufen die Werkzeuge, die im Alltag gebraucht werden, längst als Cloud-Dienste im Ausland: Dokumentation in Confluence, Auswertungen in SaaS-Tools, zunehmend Sprachmodelle, denen man gerne den eigenen Kontext geben würde.

Heute wird dieser Konflikt organisatorisch gelöst. Sensible Angaben wie Namen, Adressen, AHV-Nummern, IBANs oder Vertragsnummern werden schlicht nicht in diese Dienste eingetragen, oder von Hand unkenntlich gemacht. Das ist mühsam, nicht überprüfbar und skaliert nicht: Es genügt eine kopierte Zeile in einem Prompt, und die Zusage ist gebrochen. Was fehlt, ist keine weitere Richtlinie, sondern eine technische Schicht, die genau diese Daten ersetzt, bevor sie die eigene Grenze überschreiten.

Entscheidend ist dabei der Zeitpunkt. Daten erst beim Anzeigen wieder einzusetzen genügt nicht: Liegen die echten Werte weiterhin beim fremden Dienst und werden nur lokal versteckt, ist nichts gewonnen. Der Austausch muss stattfinden, solange die Daten noch im eigenen Einflussbereich sind. Diese Ausgangslage ist abgeleitet von der Challenge Beschreibung (siehe Anhang 2).

## Lösungsansatz

Wir bauen die fehlende Schicht dort ein, wo jeder Dienst gleich aussieht: auf dem Netzwerkweg. “Seniors in the Middle” (SITM) ist ein Man-in-the-Middle-Proxy, der als Grenzposten zwischen den Geräten einer Organisation und allem Fremden steht. Der Name ist die Selbstbeschreibung: Wir sitzen bewusst in der Mitte, und zwar auf der eigenen Seite der Grenze.

### Der Proxy als Grenze

Ein Gerät wird auf den Proxy konfiguriert und vertraut dessen CA. Damit sieht der Proxy jede ausgehende Anfrage, auch die verschlüsselten: Ein CONNECT-Tunnel wird terminiert, für den Zielhost wird im Moment des Verbindungsaufbaus ein Zertifikat ausgestellt, und der Klartext liegt im Proxy vor. Das Ersetzen passiert dadurch vor dem Verlassen der Grenze und nicht im Browser des Betrachters. Der Ansatz ist bewusst nicht auf ein Werkzeug zugeschnitten: Confluence, ein Ticketsystem und ChatGPT laufen über denselben Weg, ohne dass pro Dienst ein eigener Konnektor gebaut werden muss.

### Nur lesen, was gelesen werden muss

Der grösste Teil des Verkehrs sind Stylesheets, Skripte, Schriften und Bilder. Diese Bodies werden nie geöffnet, sondern nach Inhaltstyp durchgereicht (passthrough). Geöffnet wird nur, was einen Body trägt, der überhaupt Personendaten enthalten kann, in der Regel JSON und Text. Ergibt die Prüfung nichts, gilt die Anfrage als clean; wird etwas gefunden, wird sie behandelt (treated). Das hält die Latenz für den Löwenanteil des Verkehrs bei null und begrenzt die Menge an Klartext, die der Proxy überhaupt anfasst.

### Erkennung mit KI statt mit Regex-Listen

Was eine Person identifiziert, steht selten in einem sauber benannten Feld, sondern mitten im Fliesstext eines Prompts. Die Erkennung übernimmt deshalb ein NLP-Modell: Microsoft Presidio auf einem deutschen spaCy-Modell findet Namen, Adressen, Organisationen und Daten über Named Entity Recognition, ergänzt um musterbasierte Erkennung mit Prüfziffernvalidierung für IBAN, AHV-Nummer, Kreditkarten und Telefonnummern. Jeder Fund kommt mit Typ, Position, Konfidenz und einer Risikoeinstufung zurück, die sich am Kontinuum von Schwartz und Solove sowie an den HIPAA-Kategorien orientiert. Damit ist entscheidbar, was ersetzt werden muss und was toleriert werden kann.

### Tokens, die aussehen wie Daten

Ein Platzhalter der Form [REDACTED\_1] zerstört den Nutzen des Zieldienstes: Ein Sprachmodell antwortet schlechter, eine Formularvalidierung schlägt fehl, ein Suchindex bricht. Wir ersetzen deshalb formattreu. Aus einer echten AHV-Nummer wird eine plausible, aber erfundene AHV-Nummer, aus “Hans Meier” wird ein anderer, konsistenter Schweizer Name, aus einer Berner Adresse eine andere Berner Adresse. Der externe Dienst arbeitet auf Daten, die für ihn vollständig sind, und erhält trotzdem keine einzige echte Angabe.

### Konsistenz über Anfragen hinweg

Damit Zusammenhänge erhalten bleiben, ist die Zuordnung stabil: Derselbe echte Wert erhält über Dokumente, Sitzungen und Dienste hinweg denselben Token. Wer im Zieldienst nach dem Token sucht, findet alle Vorkommen; wer über ein Dokument mit zwei Personen argumentiert, behält zwei unterscheidbare Personen. Genau diese Stabilität macht Suche im tokenisierten Bestand überhaupt möglich: Die Suchanfrage durchläuft denselben Proxy und wird auf demselben Weg tokenisiert, bevor sie den Dienst erreicht.

### Der Tresor bleibt hier

Die Tabelle von Token zu echtem Wert verlässt den Proxy nie. Sie ist der einzige Ort, an dem die echten Daten noch stehen, und sie liegt auf einem Server in der Schweiz. Der fremde Dienst hält

ausschliesslich Tokens; selbst ein vollständiger Abfluss seiner Datenbank gibt keine Personendaten preis.

### **Rehydrierung nur für Berechtigte**

Auf dem Rückweg werden die Tokens in der Antwort wieder durch die echten Werte ersetzt, und zwar nur für Clients, die den Proxy benutzen dürfen und dessen CA vertrauen. Wer keinen Zugang hat, sieht die Tokens, denn das ist der wahre Inhalt des Dienstes. Re-Identifikation ist damit eine Berechtigung an unserer Grenze und keine Eigenschaft der Daten.

### **Sichtbarkeit als Teil des Produkts**

Eine Schicht, die man nicht sieht, wird nicht geglaubt. Der Proxy sendet jeden Schritt als Ereignis an ein Dashboard: welche Anfrage beobachtet wurde, was gefunden wurde, wie der Body nach der Ersetzung aussah, was tatsächlich hinausging, was zurückkam und was der Client zu sehen bekam. Damit ist an einer Wand nachvollziehbar, dass die echten Werte die Grenze nie überschritten haben.

## Implementierung

- Abfangender Forward-Proxy in .NET 10: absolute-form HTTP sowie HTTPS über CONNECT mit eigener CA, die beim ersten Start erzeugt wird und pro Zielhost ein Zertifikat ausstellt. Clients beziehen die CA unter /ca.crt und die Autokonfiguration unter /proxy.pac.
- Drei getrennte Listener mit je einer Rolle: Proxy-Verkehr, derselbe Proxy innerhalb von TLS, und die WebAPI mit dem Telemetrie-Stream. Der API-Port leitet nichts weiter, damit der Port des Dashboards nicht als offener Proxy missbraucht werden kann; der Prozess startet gar nicht erst, wenn die Ports kollidieren.
- Erkennung als eigener Python-Service, angebunden über einen Unix-Socket mit einem längenpräfixierten JSON-Protokoll. Das trennt die Laufzeiten sauber: Das Modell lebt in Python, der Datenpfad in .NET, und beide laufen im selben Container ohne Netzwerk-Hop.
- Formattreue Ersetzung über Faker mit deutschsprachigem Locale, gesteuert über eine Typ-Tabelle, die jeden erkannten PII-Typ auf Informationsart, Risikostufe und Ersetzungsstrategie abbildet.
- Telemetrie über SignalR: eine begrenzte Queue, ein Hintergrund-Reader, Reihenfolgegarantie pro Austausch und Verwerfen der neuesten Ereignisse unter Last, damit eine langsame Ansicht nie eine Anfrage bremst. Das Wire-Protokoll ist auf beiden Seiten typisiert und wird im Browser gegen Schemas validiert.
- Dashboard als React-SPA, das zur Laufzeit konfiguriert wird und keine eigene Logik enthält: keine Erkennung, keine Ersetzung, keine Policy. Es zeichnet ausschliesslich, was der Proxy gemeldet hat. Der Zugang dazu läuft über einen Login gegen die WebAPI, denn was das Dashboard zeigt, ist der entschlüsselte Verkehr.
- Demo-Browser für Windows (WPF und WebView2), der den Proxy und dessen CA nur im eigenen Prozess vertraut. Damit ist eine Vorführung möglich, ohne auf einem fremden Notebook eine Root-CA im Betriebssystem zu installieren.
- Integrationsumgebung, die das Backend-Image unverändert betreibt, mit einem Sender, einem Empfänger und einer eigenen CA-Kette. Sie misst die zwei Zahlen, auf die es ankommt: Der Client muss jede Angabe unverändert zurückerhalten, und der Empfänger darf keine einzige echte Angabe gesehen haben.
- Testdaten sind Schweizer Fixtures mit korrekten Prüfziffern bei AHV-Nummern und IBANs, damit eine Erkennung, die validiert, nicht an Lorem Ipsum vorbeiläuft.
- Secure: Kein Schlüsselmaterial im Image, die CA in einem gemounteten Volume, Secrets ausschliesslich über Umgebungsvariablen. Zugang zur API über JWT, der Telemetrie-Stream prüft zusätzlich den Origin des WebSocket-Handshakes selbst, weil ein Browser darauf weder CORS noch Preflight anwendet.
- CI/CD über GitHub Actions: Build, Lint, Typecheck, Tests und Container-Images pro Komponente, Deployment nach Azure Container Apps.

# Technischer Aufbau

## Bausteinsicht

Aalla Figure 1 zeigt blablbala..

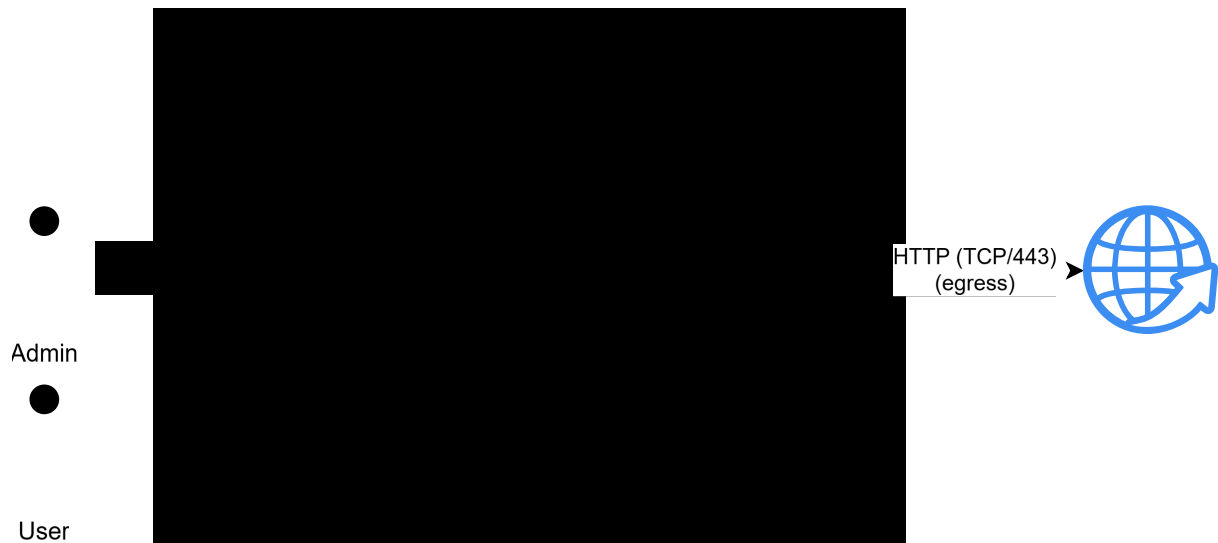


Figure 1: Die strukturelle Ansicht des Software Systems.

## Laufzeitsicht

Aalla Figure 2 zeigt blablbala..

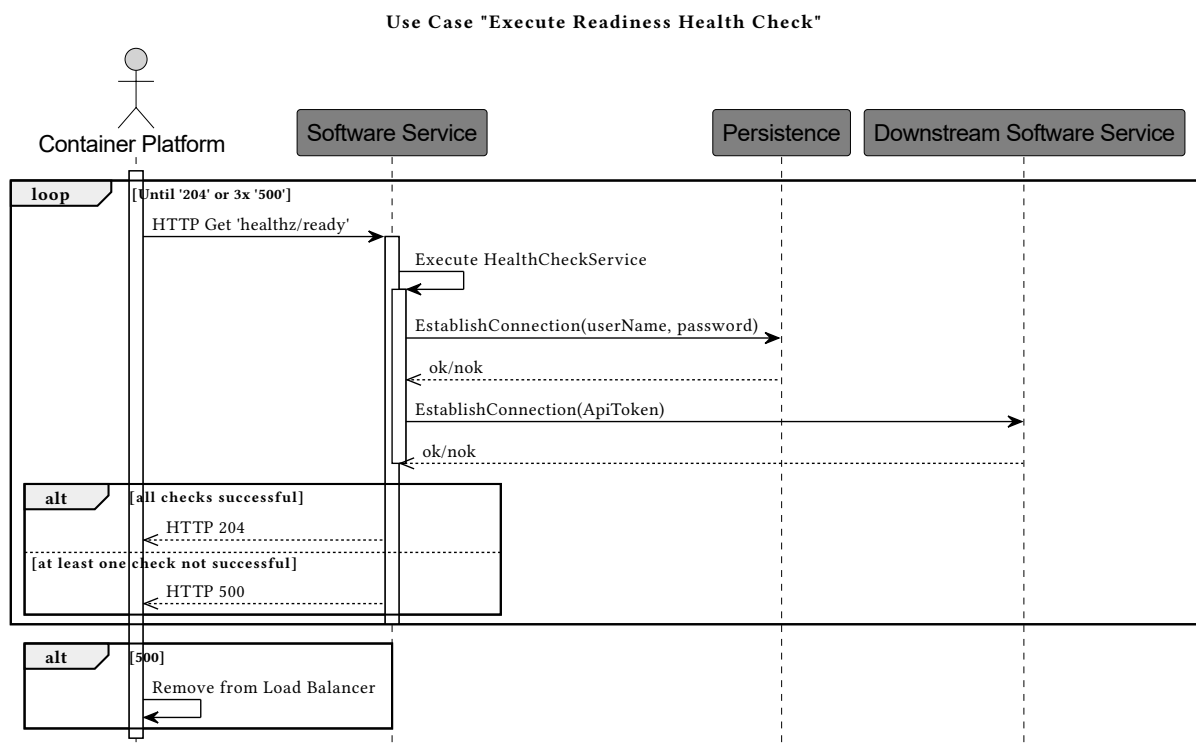


Figure 2: Das Software System zur Laufzeit.

## Verteilungssicht

Die Figure 3 zeigt blablbala..



Figure 3: Das Software System installiert auf der Produktion.

## Technologien und Frameworks

| Bereich       | Eingesetzt                                                                                                 |
|---------------|------------------------------------------------------------------------------------------------------------|
| Proxy und API | .NET 10, ASP.NET Core, Kestrel, YARP, SignalR, JWT-Bearer, OpenAPI und Swagger                             |
| PII-Erkennung | Python 3.14, Microsoft Presidio, spaCy (de_core_news_lg), Faker, Pydantic Settings, tiktoken               |
| Interprozess  | Unix Domain Sockets, eigenes längenpräfixiertes JSON-Protokoll                                             |
| Frontend      | React 18, TypeScript, Vite, Valibot, SignalR-Client, nginx                                                 |
| Demo-Client   | WPF, WebView2, .NET 10                                                                                     |
| Tests         | MSTest auf Microsoft.Testing.Platform, Vitest und Testing Library, Integrationsumgebung mit Docker Compose |
| Qualität      | ESLint, Stylelint, Prettier, Knip, EditorConfig, Nullable Reference Types                                  |
| Betrieb       | Docker, GitHub Actions, GitHub Container Registry, Azure Container Apps                                    |

## Abgrenzung / Offene Punkte

Wir zeigen die Schicht selbst, nicht eine fertige Betriebslösung. Bewusst ausserhalb des Hackathon-Umfangs liegen:

- Der Tresor ist im aktuellen Stand nicht dauerhaft persistiert. Für den produktiven Einsatz braucht es eine verschlüsselte Ablage mit Backup, denn ein verlorener Tresor macht den Bestand beim fremden Dienst unwiderruflich unlesbar.

- Berechtigungen zur Re-Identifikation sind heute binär: Wer über den Proxy geht, sieht die echten Werte. Rollen, feldgenaue Freigaben und ein revisionssicheres Protokoll darüber, wer wann welchen Token aufgelöst hat, sind der nächste Schritt.
- Felder, die der Zieldienst für sich selbst braucht, etwa die Login-Mail oder ein Benutzername in einer URL, dürfen nicht tokenisiert werden. Nötig ist eine Ausnahmeliste pro Dienst; heute ist die Entscheidung rein inhaltsbasiert.
- Anhänge, Bilder und Benachrichtigungen sind nicht abgedeckt. Ein PDF oder ein Screenshot trägt dieselben Daten und braucht einen eigenen Weg, ebenso Vorgänge, die der Dienst selbst auslöst und die uns gar nie passieren, etwa serverseitig verschickte E-Mails.
- Erkennungsqualität ist eine Abwägung. Ein übersehener Wert verlässt die Grenze, ein falsch positiver Fund macht Inhalte unbrauchbar. Wir messen das heute an eigenen Fixtures; belastbare Precision- und Recall-Zahlen auf echten Korpora fehlen, ebenso eigene Recognizer für weitere Schweizer Identifikatoren.
- Zertifikatsverteilung bleibt eine Betriebsaufgabe. Das Vertrauen in die CA ist ein Schritt im Betriebssystem, und Clients mit Certificate Pinning lassen sich grundsätzlich nicht abfangen.
- Der Proxy ist als Proof of Concept unauthentifiziert und ohne Zielbeschränkung. Für einen Einsatz ausserhalb einer kontrollierten Umgebung fehlen Authentifizierung, Zielrestriktionen, Verbindungslimits und Schutz vor Zugriffen auf interne Adressen.
- Skalierung und Latenz unter Last sind nicht vermessen. Die Erkennung ist der teuerste Schritt im Pfad und wäre der erste Kandidat für Caching und horizontale Verteilung.

## Literatur

- Natron Tech, Challenge “Swiss Data Airlock”, BärnHäckt 2026 (siehe Anhang 2).
- Microsoft Presidio, Data Protection and De-identification SDK: <https://microsoft.github.io/presidio/>
- spaCy, Industrial-Strength Natural Language Processing: <https://spacy.io/>
- Schwartz, P. M. und Solove, D. J. (2011): The PII Problem: Privacy and a New Concept of Personally Identifiable Information. New York University Law Review, Band 86, Seiten 1814 bis 1894.
- U.S. Department of Health and Human Services: Methods for De-identification of PHI: <https://www.hhs.gov/hipaa/for-professionals/special-topics/de-identification/>
- YARP, Reverse Proxy für .NET: <https://microsoft.github.io/reverse-proxy/>
- RFC 9110, HTTP Semantics, Abschnitt 9.3.6 (CONNECT): <https://www.rfc-editor.org/rfc/rfc9110>

## Anhang

- Anhang 1: Repository mit Quellcode, Dokumentation und Pitch-Material: <https://github.com/baernhaeck/SeniorsInTheMiddle>
- Anhang 2: Challenge-Beschreibung “Swiss Data Airlock” von Natron Tech. <https://www.bernhackt.ch/challenges/2026-swiss-data-airlock>

## Challenge Beschreibung

### KONTEXT UND HINTERGRUND

Wir betreiben Infrastruktur für unsere Kundschaft, und ein Teil unseres Jobs ist, sorgfältig mit ihren Daten umzugehen.

Bei manchen Kunden geht das so weit, dass gewisse Daten die Schweiz gar nicht verlassen dürfen, und zwar so, dass ein ausländischer Anbieter sie technisch nie zu Gesicht bekommt. Nicht weil ein Gesetz das pauschal verbietet, sondern weil wir es ihnen so zugesagt haben.

Das wird unangenehm, weil viele gute Tools heute in der Cloud von Drittanbietern laufen, deren Server im Ausland stehen. Für manche gibt es zwar eine Schweizer Region, aber das deckt längst nicht alles ab, und der Anbieter bleibt am Ende ein ausländisches Unternehmen. Für ein verlässliches ‘die Daten landen da nie’ reicht das nicht.

Und es bleibt nicht bei einem einzelnen Tool. Besonders heikle Daten wie Kontaktinformationen, finanzielle Angaben oder Vertragsnummern können wir nicht in Dokumentationstools wie Confluence ablegen. Dasselbe bei KI-Diensten, denen wir gerne unseren Kontext geben würden, aber ohne die sensiblen Daten.

Überall dieselbe Frage, und überall fehlt uns dasselbe: eine Schicht, die genau diese Daten ersetzt, bevor sie uns verlassen.

### BESCHREIBUNG DES PROBLEMS

Baut uns eine Schicht, die bestimmte Daten tokenisiert, bevor sie einen Dienst ausserhalb unserer Kontrolle erreichen, und sie für berechtigte Personen lokal wieder einsetzt. Nach aussen sind nur noch bedeutungslose Tokens sichtbar, für uns sieht alles normal aus, mit den echten Werten. Die Zuordnung von Token zu echtem Wert bleibt bei uns, auf einem Server in der Schweiz.

Das Ganze ist bewusst allgemein gedacht. Wir haben zwar konkrete Lösungen im Kopf, möchten die Challenge aber offen lassen.

Als Gedankenstütze haben wir zwei Beispiele: Ein Cloud-Tool wie Confluence: Wir schreiben echte Namen, Adressen und Vertragsnummern hinein, beim Anbieter landen nur Tokens, im Browser sehen wir wieder die echten Werte.



Ein KI-Dienst oder MCP-Server: Ein externes Modell arbeitet mit unserem Kontext, sieht statt der echten Personendaten aber nur Tokens, und die Antwort wird bei uns wieder aufgelöst.

Wichtig ist in beiden Fällen der Zeitpunkt. Es reicht nicht, die Daten erst beim Anzeigen oder beim Empfang der Antwort wieder einzusetzen. Sie müssen ersetzt werden, bevor sie unsere Grenze überschreiten. Wenn die echten Daten weiterhin beim fremden Dienst liegen und nur lokal versteckt werden, ist die Aufgabe nicht gelöst.

Wenn das mal steht, kommen die spannenden Fragen: Wie sucht man, wenn überall nur noch Tokens stehen? Wie bleibt dieselbe Angabe über viele Dokumente oder Anfragen hinweg konsistent maskiert, damit Zusammenhänge erhalten bleiben? Was ist mit Anhängen, Kommentaren, Benachrichtigungen? Was mit Feldern, die der Dienst für sich selbst braucht, etwa die Login-Mail, die man nicht einfach durch einen Token ersetzen kann? Und wer darf überhaupt re-identifizieren?

Wie ihr das baut, ist euch überlassen. Wir haben ein paar Ideen, aber wir sind gespannt auf eure.